操作系统

面试资料复习

目录

一、 操作系统 4 个特征？ 2
二、 OS 基本概念 2
三、 进程和程序的区别 2
四、 进程的特征 2
七、 并发与并行 3
八、 调度队列 3
九、 调度程序 3
十、 上下文切换 3
十一、 同步、异步、阻塞、非阻塞的概念 3
十二、 进程与线程 4
十三、 引入线程的目的（为什么有了进程还要有进程） 4
十四、 用户线程和内核线程 4
十五、 用户线程与核心线程的映射模型 4
十六、 进程间的通信方式 4
十七、 进程的调度 4
十八、 死锁和饥饿 5
十九、 死锁特征 5
二十、 死锁的处理方法 5
二十一、 地址绑定的方法 5
二十二、 逻辑地址和物理地址 5
二十三、 链接的方式 5
二十四、 碎片 5
二十五、 内存分配方式 5
二十六、 分段和分页的区别 7
二十七、 虚拟内存 7
二十八、 页面置换 7
二十九、 抖动 7
三十、 哲学家就餐的解决办法 7
三十一、 文件物理空间的分配方法： 8
三十二、 处理中断的过程： 8
三十三、 中断和陷阱(trap)的区别 8
三十四、 磁盘调度算法 9
三十五、 操作系统的双重模式 9
三十六、 系统调用 9
三十七、 向操作系统传递参数的几种方式 9

一、操作系统 4 个特征？

并发：宏观上同时发生，微观上交替发生

共享：共享指资源共享，指系统中的资源可供内存中多个并发执行的进程共同使用

虚拟：虚拟是指把一个物理上的实体变为若干逻辑上的对应物，物理实体是实际存在的，而逻辑对应物是用户感受到的

异步：进程的执行不是一贯到底的，而是走走停停，以不可预知的速度向前推进，这就是进程的异步

二、OS 基本概念

用户态：执行用户程序的状态

内核态：执行内核程序的状态

三、进程和程序的区别

程序本身不是进程，程序只是被动实体，而进程是活动实体（当一个可执行文件被装入内存时，一个程序才能成为进程）

四、进程的特征

(1)并发性：多个进程实体同存于内存中，能在一段时间内同时执行

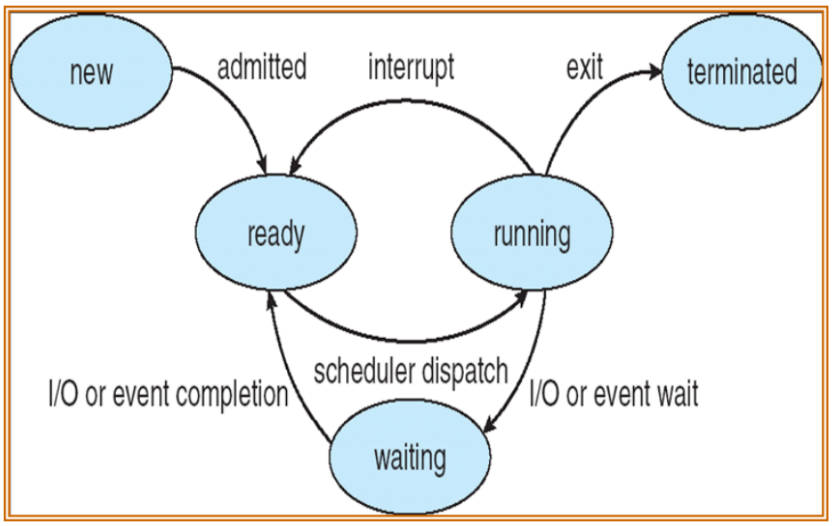
(2)动态性：进程由创建而产生，由调度而执行，因得不到资源而暂停执行，由撤销而消亡。

(3)独立性：进程实体是一个能独立运行的基本单位，同时也是系统中独立获得资源和独立调度的基本单位。

(4)异步性：进程按各自独立的、不可预知的速度向前推进。

(5)结构特征：从结构上看，进程实体由程序段、数据段以及进程控制块组成。

五、进程的状态以及转换



五个状态：新的（new），就绪（ready），运行（running），等待（waiting），终止（terminated）

*状态间转换条件

一次只有一个进程可在一个处理器上运行，但是多个进程可处于就绪或等待状态

六、PCB 及其作用

PCB是系统为了管理进程设置的一个专门的数据结构，用它来记录进程的外部特征，描述进程的运动变化过程。系统利用 PCB 来控制和管理进程。PCB 是进程存在的唯一标志

操作系统通过 PCB 而感知进程的存在

七、并发与并行

并发：在某一时间段内任务交替执行。

并行：多个任务同时执行。

八、调度队列

①作业队列（Job queue）：系统中的所有进程

②就绪队列（Ready queue）：驻留在内存中，就绪的等待运行的进程

③设备队列（Device queue）：等待 I/O 设备的进程

九、调度程序

长期调度程序又称为作业调度程序，从缓冲池中选择进程，并装入内存以准备执行。长期调度程序执行得并不频繁，控制多道程序设计的程度，还能平衡 IO 与 CPU 的利用率。

短期调度程序又称为CPU调度程序，从准备执行的进程中选择进程，并为之分配 CPU。短期调度程序要频繁地为 CPU 选择新进程，所以必须要快，它影响着 CPU 的利用率和与系统吞吐量。

中期调度程序的核心思想是能将进程从内存（或从 CPU 竞争）中移出，从而降低多道程序设计的程度。进程可换出，并在后来可被换入，一种交换（swapping）的方案。

十、上下文切换

将 CPU 切换到另一个进程需要保存当前进程的状态并恢复另一个进程的状态，这一任务称为上下文切换。

CPU在进程间的切换由中断或系统调用引起，需要保存状态至原进程的PCB，然后再从新进程的 PCB 中获取状态。

上下文切换时间与硬件支持密切相关。

上下文切换发生的场所：多任务处理（进程并发性）、中断处理（转中断处理程序）、用户态切换（切换到内核态执行内核程序）

十一、同步、异步、阻塞、非阻塞的概念

同步调用一旦开始,调用者必须等到方法调用返回后,才能继续后续的行为

异步调用更像一个消息传递,一旦开始,方法调用就会立即返回,调用者就可以继续后续的操作

阻塞：当试图对该文件描述符进行读写时，如果当时没有数据可读，或者暂时不可写，程序就进入等待状态，直到有东西可读或者可写为止

非阻塞：如果没有数据可读，或者不可写，读写函数马上返回，而不会等待

十二、进程与线程

1、进程是资源分配最小单位，线程是程序执行的最小单位

2、进程有自己独立的地址空间，每启动一个进程，系统都会为其分配地址空间，建立数据表来维护代码段、堆栈段和数据段，线程没有独立的地址空间，它使用相同的地址空间共享数据

3、CPU 切换一个线程比切换进程花费小

4、创建一个线程比进程开销小

5、线程之间通信更方便，同一个进程下，线程共享全局变量，静态变量等数据，进程之间的通信需要以通信的方式（IPC）进行；（但多线程程序处理好同步与互斥是个难点）

6、多进程程序更安全，生命力更强，一个进程死掉不会对另一个进程造成影响（源于有独立的地址空间），多线程程序更不易维护，一个线程死掉，整个进程就死掉了（因为共享地址空间）

十三、引入线程的目的（为什么有了进程还要有进程）

OS 中进程的引入是为了使多个程序并发执行，改善资源利用率，提高系统的吞吐量；OS 中引入线程是为了减少程序并发执行时所付出的时空开销，使 OS 具有更好的并发性。

十四、用户线程和内核线程

用户级线程仅存在于用户空间中。对于这种线程的创建、 撤消、切换、线程之间的同步与通信等功能，都无须利用系统调用来实现，而是通过用户级线程库来实现。用户线程受内核支持，而无需内核管理。

内核线程由操作系统直接支持和管理。

十五、用户线程与核心线程的映射模型

1、多对一模型

多对一模型将许多用户级线程映射到一个内核线程。线程管理是由线程库在用户空间进行的，因而效率比较高。但是如果一个线程执行阻塞了系统调用，那么整个进程会阻塞。因为任意时刻只有一个线程能访问内核，多个线程不能并行运行在多处理器上。（可创建任意多用户线程，不并发，不并行）

2、一对一模型

一对一模型将每个用户线程映射到一个内核线程。它提供了比多对一模型更好的并发性，也允许多个线程能并行地运行在多处理器系统上。（限制线程数量，并发， 并行）

3、多对多模型

多对多模型多路复用了许多用户线程到同样数量或更小数量的内核线程上。开发人员可以创建任意多的用户线程，并且相应内核线程能在多处理器系统上并发执行。而且，当一个线程执行阻塞系统调用时，内核能调度另一个线程来执行。（可创建任意多用户线程，并发，并行）

十六、进程间的通信方式

1、**管道（Pipes）**：管道是一种单向通信机制，通常用于在父进程和子进程之间传递数据。有两种类型：无名管道和命名管道。无名管道是父子进程之间的通信方式，而命名管道（FIFO）可用于任意两个进程之间的通信。

2、**消息队列（Message Queues）**：消息队列是一种进程间通信机制，允许多个进程通过发送和接收消息来进行数据交换。消息队列通常用于进程之间的异步通信。

3、**共享内存（SharedMemory）**：共享内存是一种高效的进程间通信方式，允许多个进程访问共享的内存区域。这使得数据可以直接在进程之间共享，而无需复制，但需要注意处理同步和互斥问题，以防止数据冲突。

4、**信号（Signals）**：信号是一种轻量级的通信方式，用于在进程之间传递异步通知。例如，进程可以向其他进程发送信号来请求它们执行某些操作。

5、**套接字（Sockets）**：套接字通常用于网络通信，但也可以在同一台计算机上的进程之间进行通信。套接字提供了一种通用的、全双工的通信机制。

6、**文件（File）**：进程可以通过读写共享的文件进行通信。这是一种简单的通信方式，但通常不适合高性能或大规模数据传输。

7、**信号量（Semaphores）**：信号量是用于进程同步的计数器，可以用来控制多个进程对共享资源的访问。

8、**消息传递（MessagePassing）**：消息传递是一种更通用的通信机制，其中进程通过发送和接收消息来交换数据。这可以包括直接进程间通信（如进程间消息传递）或通过中间件实现的分布式通信。

十七、进程的调度

1、先来先服务

2、时间片轮转

3、最短作业优先

4、最短剩余时间优先

5、优先级调度

6、多级反馈队列调度

十八、死锁和饥饿

(1)**死锁**：一组处于等待（阻塞）状态的进程，每一个进程持有其他进程所需要的资源，而又等待使用其他进程所拥有的资源，致使这组进程互相等待，均无法向前推进。另一种定义：当一组进程中每个进程都在等待一个事件，而这一事件只能由这一组进程的另一个进程引起时，这组进程处于死锁状态。

(2)**饥饿**：就绪进程长时间得不到调度是处于等待状态，而不是死锁中的互相等待。若信号量的等待队列按照 LIFO 或优先级管理，则可能导致饥饿。

十九、死锁特征

(1)mutual exclusion 互斥：至少一个资源要求互斥地共享

(2)hold and wait 占有并等待：一个进程至少占有一个资源并等待另一资源，该资源为其它进程所占有

(3)no preemption 非抢占：资源不能被抢占，只能进程完成任务后自动释放

(4)circular wait 循环等待：互相等待形成一个环

二十、死锁的处理方法

(1)使用协议预防或避免死锁

死锁预防 prevention 是一组方法，需要确定至少一个必要条件不成立

死锁避免 avoidance 要求操作系统事先得到有关进程申请使用资源的额外信息。当进程

申请资源时，若发现满足该资源的请求可能导致死锁发生，则拒绝该申请。

(2)允许进入死锁状态，检测并加以恢复

(3)忽视死锁问题：大多数系统使用，因为死锁发生并不频繁，预防、避免和恢复耗费太大。

二十一、地址绑定的方法

（1）编译时：编译时就知道进程将在内存中的驻留地址，故在编译时生成绝对代码。例如继承驻留在内存地址 R 处，则生成的编译代码就从 R 处开始向后扩展，若开始地址发生改变，需要重新编译。

缺点：不利于程序在内存中的浮动，不支持虚拟内存机制。

（2）加载时：绑定在加载时完成。若开始地址改变，只需要重新加载用户代码来引入改变值。

缺点：不利于程序的浮动，不支持虚拟内存机制。

（3）执行时：进程在执行时可以从一个内存段移动到另一个内存段，则绑定必须延迟到执行时完成。要有硬件支持，支持虚拟内存机制。

二十二、逻辑地址和物理地址

（1）逻辑地址空间：CPU 生成的地址称为逻辑地址（也称虚拟地址），由程序所生成的所有逻辑地址的集合叫逻辑地址空间。

（2）物理地址空间：内存单元的地址叫做物理地址，与程序的逻辑地址空间所对应的为物理地址空间。

二十三、链接的方式

1、静态链接：

定义：运行前将所有程序模块链接在一起，形成可执行文件，运行时直接装入内存。

优点：运行速度快。

缺点：链接及装入过程费时，可能装入用不到的模块，不利于模块的升级。

2、动态链接：

定义：运行时链接，且仅链接需要的模块。

优点：便于模块的升级，减少了链接的时间，节省内存空间。缺点：运行速度慢。

二十四、碎片

定义：无用的空闲内存空间

分类：内碎片和外碎片。内碎片存在于进程所在内存区域内部，是属于该进程的。外碎片不属于任何进程，在所有进程外。

降低外碎片的方法：

a.紧缩:移动内存内容，使空闲空间形成一整块。需要重定向是动态的并且在运行时采用。

b.允许物理地址空间不连续。

二十五、内存分配方式

1、**连续内存分配**：为每个调入内存的进程分配一段连续的内存区域。利用重定位寄存器和界限地址寄存器实现。

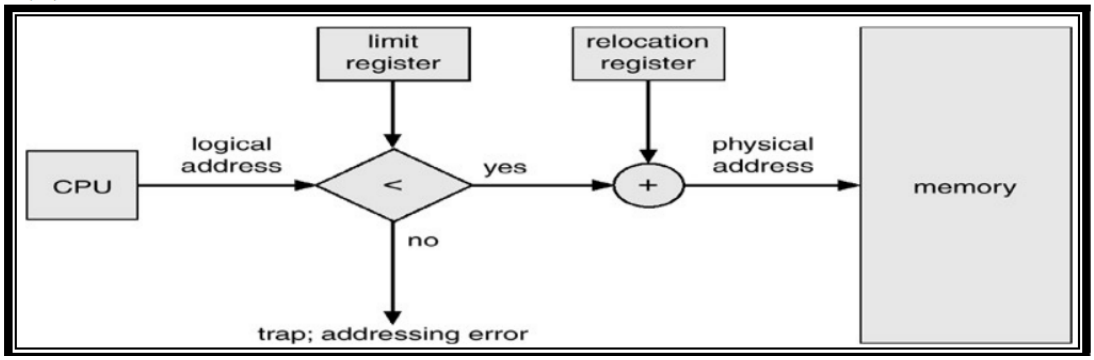
分区分配算法：为进程在空闲分区表中寻找合适的分区，产生外碎片

a.首次适应：分配找到的第一个足够大的可用分区，寻找的起始位置可以是表头，也可以是上次首次适应结束时的位置。

b.最佳适应：分配足够大的里面最小的孔，可以产生最小的剩余孔。必须要遍历整个表，除非这个表按照大小排序过。

c.最差适应：分配最大的表，可以产生最大的剩余孔。也必须遍历整个表，除非这个表按大小排序过。

d.（补充）next-fit：从上次找到合适孔的位置继续往后找此次合适的孔。



2、**分页**：一个作业在内存的各个页面可分配到不同帧中，但每个页面是连续的。用户视角的内存和实际的物理内存的分离。内存分帧，作业分页，帧的大小和页的大小相同。内存分配以页为单位。会产生**内碎片**。

1）基本定义：

a、页（page）：将逻辑内存分成的固定大小的块

b、帧（frame）：将物理内存分成的固定大小的块。

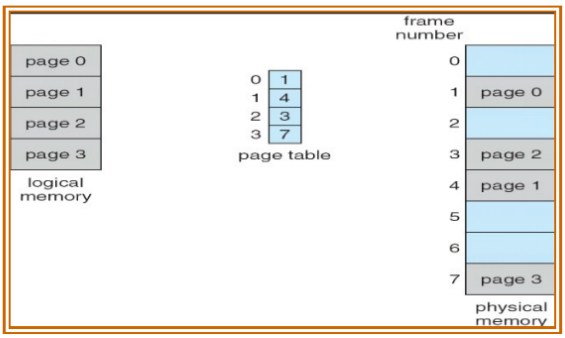
c、页表：将页和帧完成映射（逻辑地址到物理地址的映射）。由于页号是从0开始按顺序排列的，所以页表中忽略页号，只留帧号。

d、逻辑地址格式：页号+页内偏移量

e、帧表（frametable）：内存分配的细节，如哪些帧已用，哪些帧可用，总帧数等。每个条目代表一个帧，表示该帧空闲或占有（被哪个进程的哪个页占用）。

地址映射：

根据页表显示，逻辑地址空间的 0 号页映射到 1 号帧，
1 号页映射到 4 号帧，
2 号页映射到 3 号帧，
3 号页映射到 7 号帧，



2）硬件支持

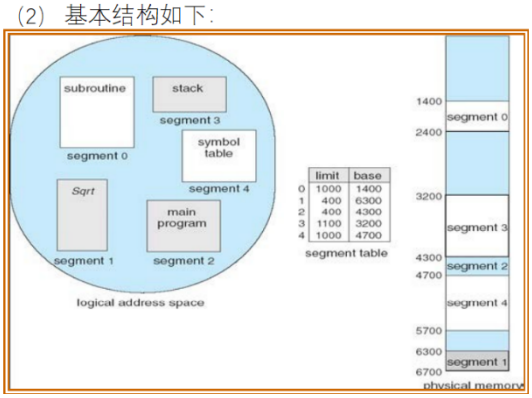
a、作为一组专用寄存器来实现：寄存器的访问速度快，所以地址变换的速度快。但是页表较大时成本太大。

b、将页表放入内存，使用页表基寄存器（Page-tablebaseregister，PTBR）指向页表，页表长度寄存器（Page-table length register，PTLR）记录页表长度 。进程未执行时页表的初始地址和长度保存在 PCB 中，执行时装入到相应寄存器中。此方法访问一个字节需要先从内存中访问页表得到物理地址，然后再次访问内存取得目的数据，一次需要两次访问内存，过于耗时。

c、使用转换表缓冲区（translation look-aside buffers ，TLB）：类似于高速缓存，是一种快速内存，TLB 中只包含页表中的一部分条目，CPU 产生逻辑地址后，先在 TLB 中查找是否有相应页号，若找到则得到了对应帧号，再根据帧号访问内存即可。若未找到页号（称为 TLB 失效），则与 2 中查找方式相同，访问两次内存， 并把对应页号和帧号放入 TLB。若 TLB 已满，则操作系统选择一个进行替换（有些 TLB 允许某些条目固定在其中，永远不被替换出去，例如一些内核代码的条目）

3、分段：支持用户视角的内存管理方案。作业分段，内存按动态分区管理。内存分配以段为单位，每段都有段名和长度。作业在内存中不连续，但是在段内连续。产生**外碎片**

(2) 基本结构如下:



二十六、分段和分页的区别

1、页是信息的物理单位，分页是为实现离散分配方式，以消减内存的外碎片，提高内存的利用率；或者说，分页仅仅是由于系统管理的需要，而不是用户的需要；段是信息的逻辑单位，它包含有一组其意义完整的信息。分段的目的是为了能更好地满足用户的需要；

2、页的大小固定且由系统决定，把逻辑地址划分为页号和页内地址两部分，是由机器硬件实现的，因为一个系统只有一种大小的页面; 段的长度不固定，决定于用户所编写的程序，通常由编译程序对源程序进行编译时，根据信息的性质来划分；

3、分页的作业地址空间是一维的，即单一的线性地址空间，程序员只须一个地址记忆符，即可表示一个地址；分段的作业地址空间是二维的，程序员在标识一个地址时，既需给出段名，又需给出段内地址

二十七、虚拟内存

（1）定义：仅把作业的一部分装入内存即可执行，具有请求调入功能和置换功能，能在逻辑上对内存空间加以扩充的一种存储器系统。

（2）思想: 根据局部性原理，我们只需要将当前要执行的那部分页或段先放入内存执行，其他先放在磁盘中。若程序想要访问的页（或段）未在内存中（称为缺页或缺段），操作系统调用请求调页（段）功能将所要访问的放入内存。若此时内存已

满，则需要利用置换功能将内存中暂时不用的页（段）调出内存，将新的页（段）调入内存。

（4）优点：使一个大程序可以在小内存中运行；用户在一个虚拟内存中编程，使得编程不再受内存容量限制；内存可以装入更多程序并发执行；提高系统的并发度和吞吐量，减少了 I/O 时间。

二十八、页面置换

1、FIFO 页置换

2、最优置换（Optimal Algorithm，OPT）

3、LRU 置换（Least-Recently-Used）

4、近似 LRU 置换：附加引用位算法、二次机会算法、增强二次机会算法

二十九、抖动

定义：进程没有“足够”的帧，需要进行页调度，而调度出去的页再次被需要，所以又很快被调回来，导致进程用于页调度的时间多于执行的时间。

解决方法：

（1）采用局部置换或优先级置换算法

（2）工作集算法

三十、哲学家就餐的解决办法

1）至多 n-1 个哲学家同时进餐

2）检测左右两边都有筷子才能进餐

3）奇数哲学家先拿左边的筷子，偶数哲学家先拿右边的筷子

三十一、文件物理空间的分配方法：

1、**连续分配**：每个文件在磁盘上占有一组连续的块，可以用第一块磁盘地址和连续块的数量来定义。

1）优点：

实现简单，随机存取速度快，效率高，适合文件内容不进行变动的情况（swap space）

2）问题：

难以进行文件扩展，需要提前声明文件大小产生外碎片

2、**链接分配**：每个文件是磁盘块的链表，磁盘块分布在磁盘的任何地方，目录块包含文件第一块和最后一块的指针。

1）优点：

实现简单，只需要首地址；没有外碎片问题；文件可以扩展，不需要提前声明文件大小

2）问题：

每个文件块都有指针，占用空间；无法实现随机读取；可靠性差，一个中间数据块中指针的丢失都会导致链的断裂

3、**索引分配**：把所有指针放到统一的索引块上，上面存放着磁盘块地址的数组，索引块的第 i

个条目指向了文件的第 i个块

1）优点：随机访问；可扩展；文件不需要提前声明大小；没有外碎片

2）问题：浪费空间

三十二、处理中断的过程：

(1)保护断点：即保存下一将要执行的指令的地址，就是把这个地址送入堆栈。

(2)寻找中断入口：根据不同的中断源所产生的中断向量，查找不同的入口地址，入口地址处存放着中断处理程序。

(3)执行中断处理程序。

(4)中断返回：执行完中断指令后，就从中断处返回到主程序，继续执行。

三十三、中断和陷阱(trap)的区别

陷阱(trap)又被称作软中断，与（硬）中断相比，软中断是软件实现的中断，也就是程序运行时其它程序对它的中断；而硬中断是硬件实现的中断，是程序运行时设备对它的中断。（硬中断是由外部事件引起的，因此具有随机性和突发性；而软中断的发生不是随机的，而是由程序安排好的）

三十四、磁盘调度算法

1、FCFS：先来先服务，根据进程请求访问磁盘的的先后次序进行调度。算法比较公平，但是因为磁臂摆动幅度大导致平均寻道距离大，效率低

2、SSTF：最短寻道时间优先算法（贪心算法）每次都选择处理距离当前磁头位置最近的待处理请求。提高了性能但不是最优。基本与 SJF 最短作业优先调度一样，因为新请求随时会抵达，所以可能导致磁头一直在一个小范围摆 动，产生饥饿

3、SCAN：又称电梯调度算法。磁臂从磁盘的一端向另一端移动，磁头经过每个柱面时处理位于该柱面上的请求。到达**另一端**时改变移动方向继续处理。避免了饥饿现象。但当磁头到达一端折返往回移动时， 在该端等待服务的请求可能很少，等待的时间也比较短，因为该区域的请求刚刚被访问过；而另一端等待服务的请求可能很多，等待的时间也可能很长

4、C- SCAN：circular SCAN，SCAN 调度变种，要求磁头单项移动，即磁头从磁盘一边移动到另一端并处理请求，当磁头移动到另一端时会马上返回磁盘开始，返回时不处理请求。提供更为均匀的等待时间，与 SCAN 相比比较公平，但需要移动更多个柱面

5、LOOK，C-LOOK：SCAN 和 C-SCAN 是使磁头在整个磁盘宽度内进行移动，但这并不容易实现。通 常，磁头只会移动到一个方向上最远的请求为止。与 SCAN 和 C-SCAN 相比提高了效率。

三十五、操作系统的双重模式

组成：用户模式（user mode）和核心模式（kernel mode），即用户模式下应用程序需要操作系统的服务时必须切换至核心模式由操作系统完成相应请求。

动机：将能引起机器损害的机器指令作为*特权指令（privileged instruction），如转换到用户模式，IO 控制，定时器管理和中断管理等，通过识别模式位保证特权指令只能由操作系统完成，保护操作系统和用户程序不受错误用户程序影响。

三十六、系统调用

操作系统内核提供一系列预定功能，通过一组称为系统调用的接口呈现给

编程人员，系统调用把应用程序的请求传给内核，系统调用相应的内核函数完成所需的处 理，将处理结果返回给应用程序。C 或 C++编写。系统调用是内核的一部分。

系统调用大致分为五大类：进程控制，文件管理，设备管理，信息维护和通信

三十七、向操作系统传递参数的几种方式

(1)通过寄存器来传递参数

(2)将参数存在内存的块和表中，将块的地址通过寄存器来传递

(3)参数通过程序压入堆栈，通过操作系统弹出